

Exercise - Coffee Machine Interface (CLI)

In this exercise, you will develop a Coffee Machine program using Python and CLI only (Command Line Interface). The system should simulate a real coffee machine that manages resources, receives payments and prepares drinks.

General Guidelines

- This exercise must be solved individually.
- Validate all user input before continuing execution.
- Use functions to separate responsibilities in the program.
- **Use OOP principles learned in class.**
- Do not print unnecessary messages or debug information.
- The program must run without warnings or runtime errors.
- The machine should continue serving customers until the user turns it off.

Program Requirements

1. Main Prompt

Prompt the user with: "What would you like? (espresso/latte/cappuccino):". The prompt should appear again after every completed action.

2. Turn Off the Machine

If the user enters "off", the program should terminate execution.

3. Print Report

If the user enters "report", display the current machine resources:

Water: 100ml

Milk: 50ml

Coffee: 76g

Money: \$2.5

4. Check Resources

Before making a drink, verify that there are enough resources. If a resource is insufficient, print the relevant message, such as:

"Sorry there is not enough water."

5. Process Coins

If resources are sufficient, prompt the user to insert coins.

Coin values:

- Quarter = \$0.25

- Dime = \$0.10
- Nickel = \$0.05
- Penny = \$0.01

Calculate the total inserted amount.

6. Check Transaction

If the inserted money is insufficient, print:

“Sorry that's not enough money. Money refunded.”

If enough money was inserted, update the machine profit.

If too much money was inserted, calculate and return change rounded to 2 decimal places.

7. Make Coffee

If the transaction succeeds:

- Deduct the required resources.
- Print a success message such as:
“Here is your latte. Enjoy!”

Submission Requirements

- Submit all Python source files.
- The program must run correctly from the terminal.
- Code must be readable, modular, and properly structured.

Grading Criteria

- Correctness. (50 pt)
- Logic (OOP). (20 pt)
- Style/PEP 8. (10 pt)
- Documentation. (5 pt)
- Efficiency. (15 pt)

Good luck